Nonlinear ARX Modeling of an Unknown Dynamic System

Circiu Patrick-Sorin
Mureșan Ionuț-Alin
Biriș Marc-Emil
Group 30311

Table of contents

01 ARX Model Structure

02 Model Identification Procedure

03 MATLAB Implementation

04 Conclusion

01

ARX Model Structure

Introduction

A **Nonlinear ARX (NARX) model** is an extension of the classical **ARX model** that can capture nonlinear relationships in a system.

- **ARX model (linear)** predicts the current output $y(k)$as a **linear combination** of past outputs and past input

$$\begin{aligned}\hat{y}(k) = &- a_1 y(k-1) - a_2 y(k-2) - \ldots - a_{na} y(k-na) \\ &+ b_1 u(k-1) + b_2 u(k-2) + \ldots + b_{nb} u(k-nb)\end{aligned}$$

- **Nonlinear ARX model** allows the output to depend on **polynomial (or other nonlinear) functions** of past outputs and inputs:

$$\begin{aligned}y(k) &= p(y(k-1), \ldots, y(k-na), u(k-1), \ldots, u(k-nb)) + e(k) \\ &=: p(d(k)) + e(k)\end{aligned}$$

where $d(k) = [y(k-1), \ldots, y(k-na), u(k-1), \ldots, u(k-nb)]^\top$ is the vector of delayed signals.

- Develop a black box for a nonlinear ARX model with unknown dynamics
- Use a polynomial nonlinear ARX to capture possible noise effects

02

Model Identification Procedure

NARX Structure

- The model predicts the output as a **polynomial function of delayed outputs and inputs**
- **Prediction:** uses actual measured past outputs
- **Simulation:** uses model-predicted past outputs recursively

Finding the Parameters:

- Compute all polynomial terms (theta) of delayed inputs and outputs, arrange them in a regression matrix, and solve a linear regression to estimate the model parameters.

$$
\begin{aligned}
y(k) &= p(y(k-1), \dots, y(k-na), u(k-1), \dots, u(k-nb)) + e(k) \\
&=: p(d(k)) + e(k)
\end{aligned}
$$

where $d(k) = [y(k-1), \dots, y(k-na), u(k-1), \dots, u(k-nb)]^\top$ is the vector of delayed signals.

One-step ahead prediction $\hat{y}$: True output sequence is known, delays vector $d(k)$ is fully available:

$$
\begin{aligned}
d(k) &= [y(k-1), \dots, y(k-na), u(k-1), \dots, u(k-nb)]^\top \\
\hat{y}(k) &= g(d(k); \hat{\theta})
\end{aligned}
$$

Simulation $\tilde{y}$: True outputs unknown, use the previously simulated outputs to construct an *approximation* $\tilde{d}(k)$ of $d(k)$:

$$
\begin{aligned}
\tilde{d}(k) &= [\tilde{y}(k-1), \dots, \tilde{y}(k-na), u(k-1), \dots, u(k-nb)]^\top \\
\tilde{y}(k) &= g(\tilde{d}(k); \hat{\theta})
\end{aligned}
$$

03

MATLAB Implementation

MATLAB Implementation

```
clc
clear
close all
load iddata-10.mat

uid = id.InputData{:};
yid = id.OutputData{:};
uval = val.InputData{:};
yval = val.OutputData{:};

u_id = uid(1:end-1);
y_id = yid(1:end-1);

u_val = uval(1:end-1);
y_val = yval(1:end-1);
```

```
function exp = exp_poly(nx, m)
    exp = [];
    for d = 0:m
        exp = [exp; exp_deg(nx, d)];
    end
end

function e_deg = exp_deg(nx, deg)
    if nx == 1
        e_deg = deg;
    else
        e_deg = [];
        for k = 0:deg
            ek = exp_deg(nx-1, deg-k);
            e_deg = [e_deg; [k*ones(size(ek,1),1), ek]];
        end
    end
end
```

MATLAB Implementation

```
function PHI = phi_poly(X, m)
    [N, nx] = size(X); % nx = na + nb
    exp = exp_poly(nx, m);
    PHI = zeros(N, size(exp,1));
    for k = 1:size(exp,1)
        PHI(:,k) = prod(X.^exp(k,:), 2);
    end
end

function [teta,Yap,PHI] = calcTeta(X,y,m)
    PHI = phi_poly(X, m);
    teta = PHI\y;
    Yap = PHI*teta;
end

function Yvalap = calcYVal(X,m,teta)
    PHI = phi_poly(X, m);
    Yvalap = PHI*teta;
end
```

```
function MSE = MSECalc(y,Yap,N)
    sum = 0;
    for i = 1:N
        sum = sum + (Yap(i) - y(i))^2;
    end
    MSE = 1/N * sum;
end
```

MATLAB Implementation

```
function X = yprediction(y, u, na, nb)
    N = length(y);
    maxn = max(na, nb);
    Ns = N - maxn;
    X = zeros(Ns, na + nb);
    for k = 1:na
        X(:, k) = y(maxn-k+1:N-k);
    end
    for k = 1:nb
        X(:, na+k) = u(maxn-k+1:N-k);
    end
end
```

One-step ahead prediction $\hat{y}$: True output sequence is known, delays vector $d(k)$ is fully available:

$$d(k) = [y(k-1), \ldots, y(k-na), u(k-1), \ldots, u(k-nb)]^\top$$
$$\hat{y}(k) = g(d(k); \hat{\theta})$$

Simulation $\tilde{y}$: True outputs unknown, use the previously simulated outputs to construct an *approximation* $\tilde{d}(k)$ of $d(k)$:

$$\tilde{d}(k) = [\tilde{y}(k-1), \ldots, \tilde{y}(k-na), u(k-1), \ldots, u(k-nb)]^\top$$
$$\tilde{y}(k) = g(\tilde{d}(k); \hat{\theta})$$

MATLAB Implementation

```
function y_sim = calc_ysim(u, y, na, nb, m, teta)
    N = length(u);
    maxn = max(na, nb);
    y_sim = zeros(N,1);
    y_sim(1:maxn) = y(1:maxn);

    for k = maxn+1:N
        x = zeros(1, na+nb);
        for i = 1:na
            x(i) = y_sim(k-i);
        end
        for j = 1:nb
            x(na+j) = u(k-j);
        end
        phi = phi_poly(x, m);
        y_sim(k) = phi * teta;
    end
end
```

One-step ahead prediction $\widehat{y}$: True output sequence is known, delays vector $d(k)$ is fully available:

$$d(k) = [y(k-1), \ldots, y(k-na), u(k-1), \ldots, u(k-nb)]^\top$$
$$\widehat{y}(k) = g(d(k); \widehat{\theta})$$

Simulation $\tilde{y}$: True outputs unknown, use the previously simulated outputs to construct an *approximation* $\tilde{d}(k)$ of $d(k)$:

$$\tilde{d}(k) = [\tilde{y}(k-1), \ldots, \tilde{y}(k-na), u(k-1), \ldots, u(k-nb)]^\top$$
$$\tilde{y}(k) = g(\tilde{d}(k); \widehat{\theta})$$

MATLAB Implementation

```
function [] = plotsval(y_sim,y_pred,y)
figure
subplot(211)
plot(y,Color='r',DisplayName='yval')
hold on
plot(y_sim,Color='g',LineStyle='--',DisplayName='ysim')
title('y simulated compared to y validation')
legend show
hold off
subplot(212)
plot(y,Color='r',DisplayName='yval')
hold on
plot(y_pred,Color='g',LineStyle='--',DisplayName='ypred')
title('y prediction compared to y validation')
legend show
end
```

```
function [] = plotsid(y_sim,y_pred,y)
figure
subplot(211)
plot(y,Color='r',DisplayName='yid')
hold on
plot(y_sim,Color='g',LineStyle='--',DisplayName='ysim')
title('y simulated compared to y id')
legend show
hold off
subplot(212)
plot(y,Color='r',DisplayName='yid')
hold on
plot(y_pred,Color='g',LineStyle='--',DisplayName='ypred')
title('y prediction compared to y id')
legend show
end
```

MATLAB Implementation

```
na = 4;
nb = 5;
m  = 2;

X_id = yprediction(y_id, u_id, na, nb);
X_val = yprediction(y_val, u_val, na, nb);

y_idd = y_id(max(na,nb)+1:end);
y_vall = y_val(max(na,nb)+1:end);

N_id = length(y_idd);
N_val = length(y_vall);

[teta_id,Yap_id,PHI_id] = calcTeta(X_id,y_idd,m);
y_pred = calcYVal(X_val,m,teta_id);
y_sim = calc_ysim(u_val, y_val, na, nb, m, teta_id);

y_simid = calc_ysim(u_id, y_id, na, nb, m, teta_id);
y_predid = PHI_id * teta_id;

plotsid(y_simid(max(na,nb)+1:end), y_predid, y_id)
plotsval(y_sim(max(na,nb)+1:end), y_pred, y_val)
```

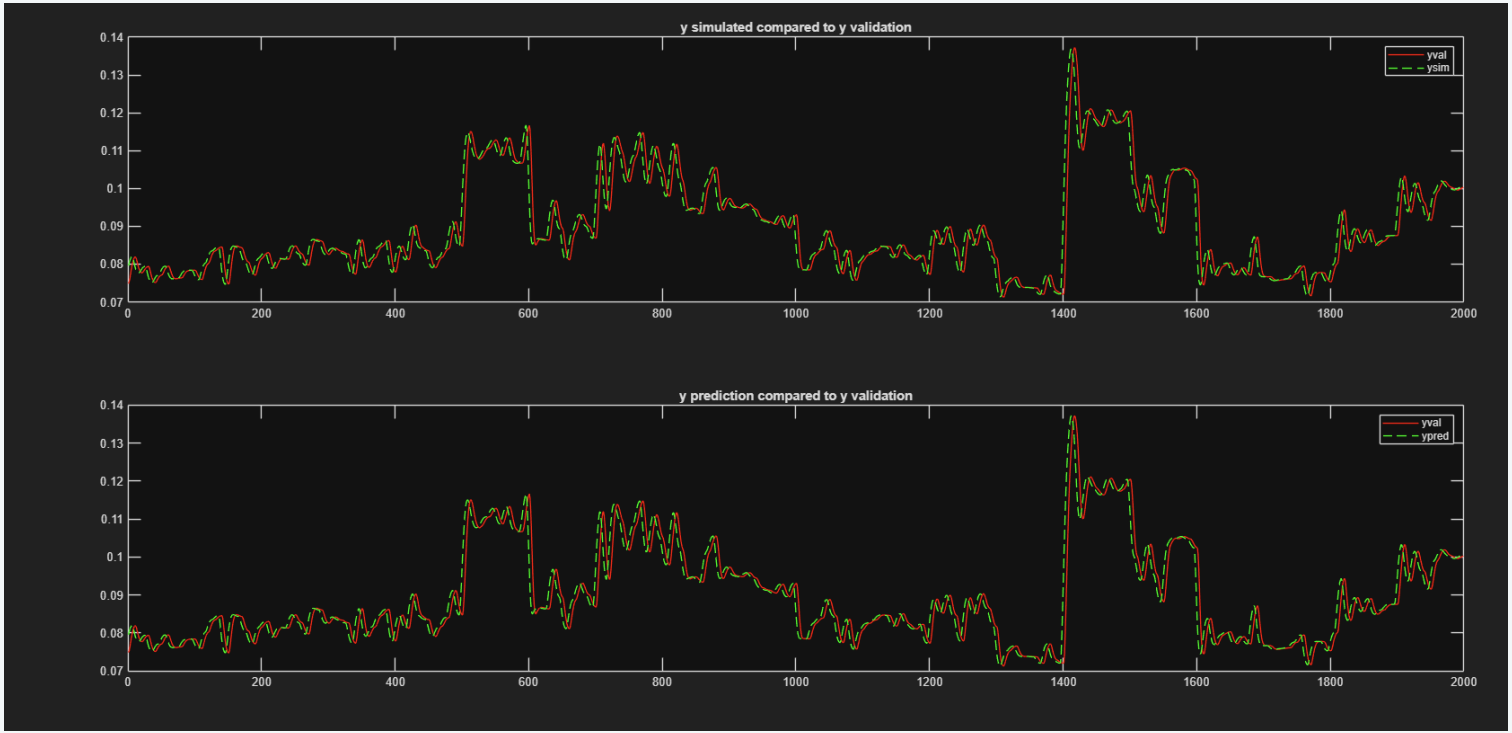
MATLAB Implementation

```
%% CALCULATING BEST na,nb and m USING MSE
MSEbun = inf;
for na = 1:5
    for nb = 1:5
        for m = 1:5

            maxn = max(na, nb);
            X_id  = yprediction(y_id,  u_id,  na, nb);
            X_val = yprediction(y_val, u_val, na, nb);

            y_idd  = y_id(maxn+1:end);
            y_vall = y_val(maxn+1:end);
            try
                teta_id = phi_poly(X_id, m) \ y_idd;
            catch
                continue
            end
            y_pred = phi_poly(X_val, m) * teta_id;
            N = length(y_pred);
            mse_val = MSECalc(y_vall, y_pred,N);
            if mse_val < MSEbun
                MSEbun = mse_val;
                nabun  = na;
                nbbun  = nb;
                mbun   = m;
            end
        end
    end
end
```

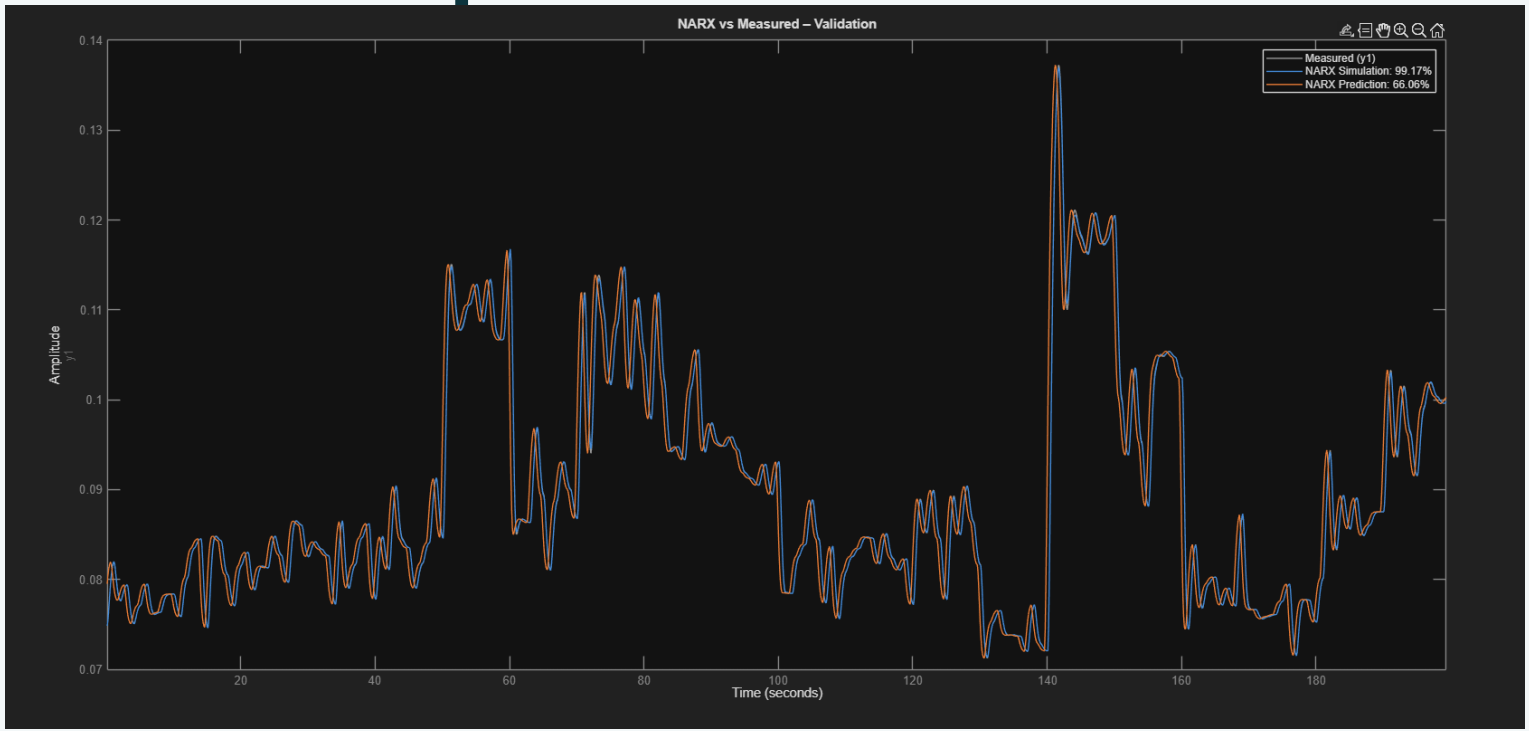
```
fprintf('MSEbun = %.6g\n', MSEbun);
fprintf('m = %d\n', mbun);
fprintf('na = %d\n', nabun);
fprintf('nb = %d', nbbun);

Ts_id = id.Ts{1};
Ts_val = val.Ts{1};

sim_id = iddata(y_simid(1:1994), u_id(1:1994), Ts_id);
pred_id = iddata(y_predid(1:1994), u_id(1:1994), Ts_id);

sim_val = iddata(y_sim(1:1994), u_val(1:1994), Ts_val);
pred_val = iddata(y_pred(1:1994), u_val(1:1994), Ts_val);

id_data  = iddata(y_id(1:1994), u_id(1:1994), Ts_id);
val_data = iddata(y_val(1:1994), u_val(1:1994), Ts_val);

figure
compare(id_data, sim_id, pred_id)
legend('Measured','NARX Simulation','NARX Prediction')
title('NARX vs Measured - Identification')
figure
compare(val_data, sim_val, pred_val)
legend('Measured','NARX Simulation','NARX Prediction')
title('NARX vs Measured - Validation')
```

MATLAB Implementation

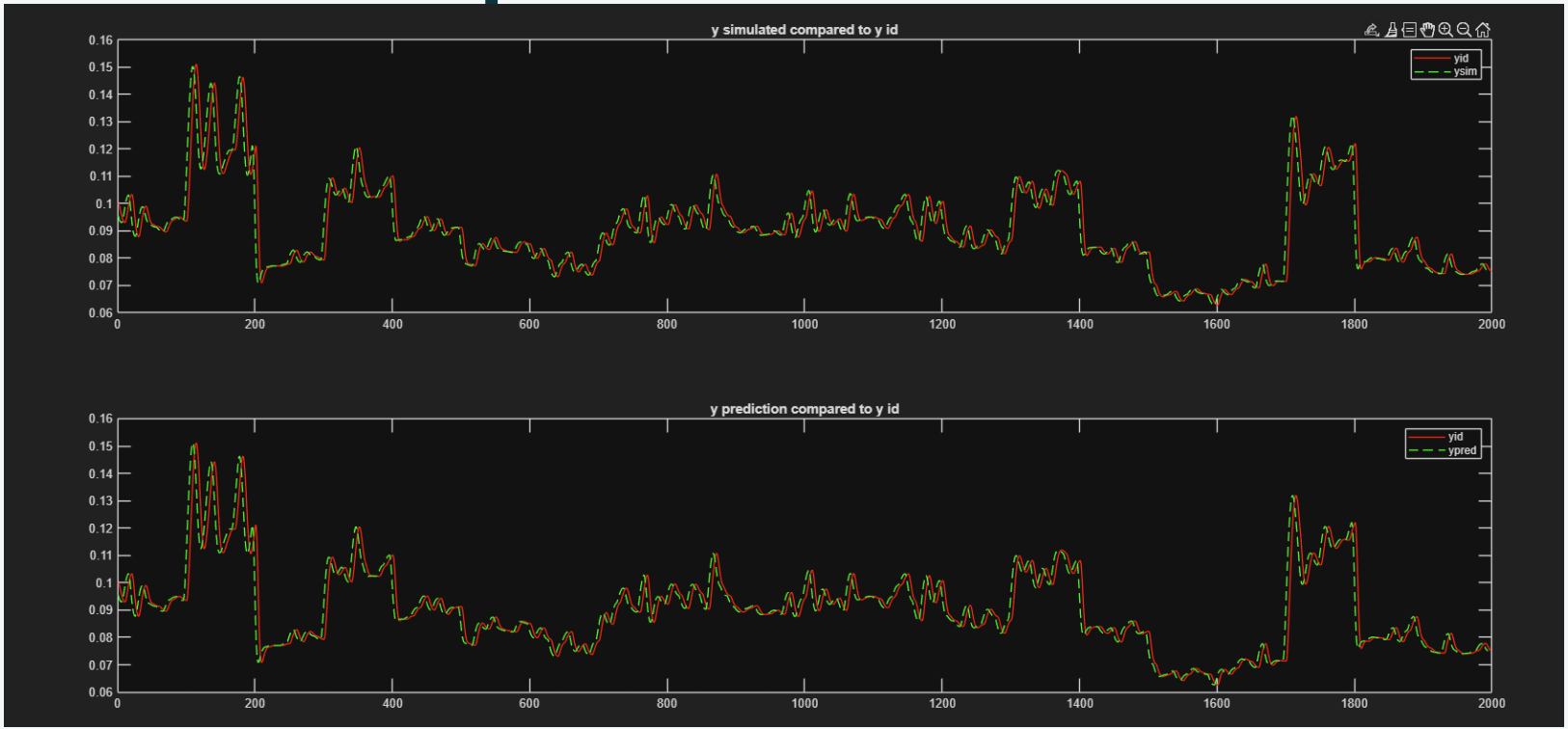
```
MSEbun = 1.34651e-10
m = 2
na = 4
nb = 5>> Press Ctrl + S
```



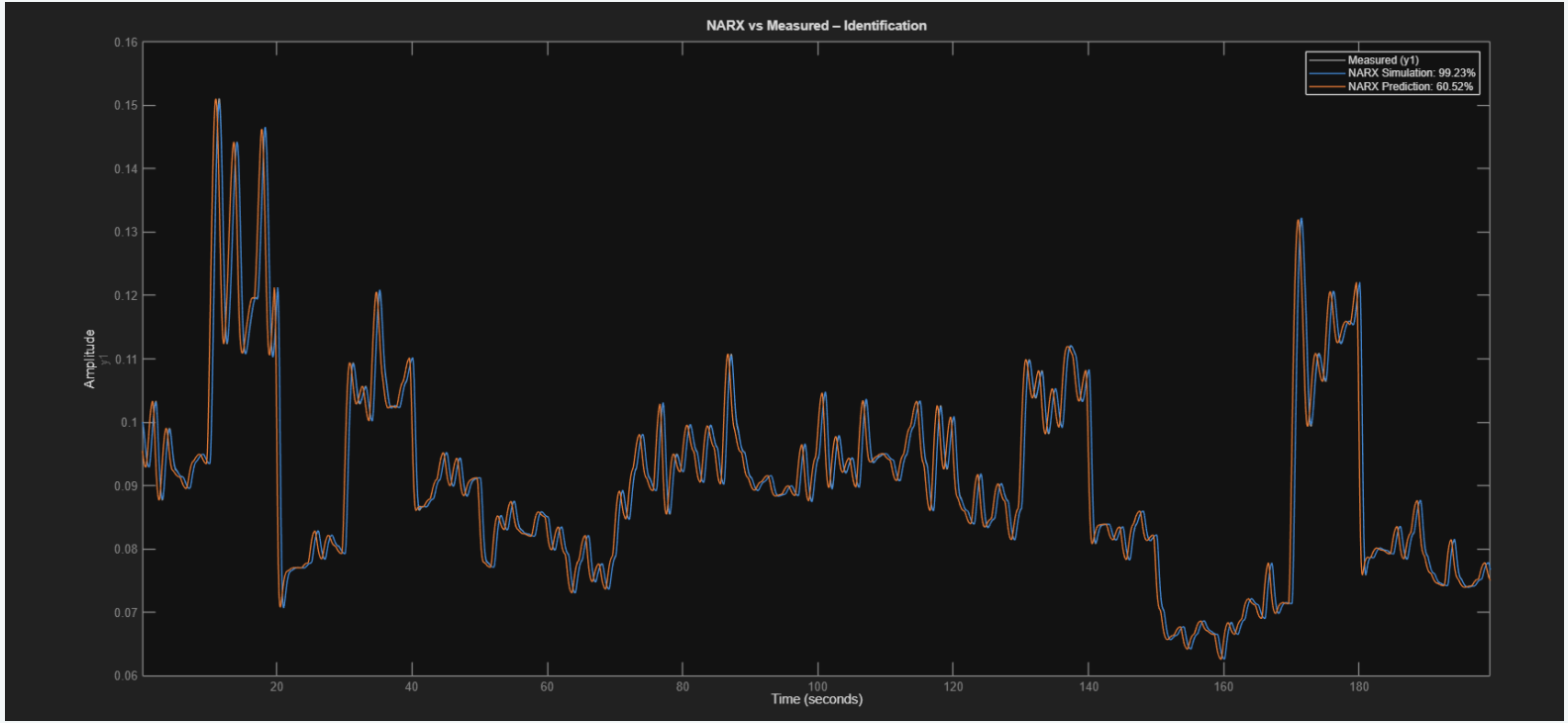
MATLAB Implementation



MATLAB Implementation



MATLAB Implementation



MATLAB Implementation

MSE	m=1	m=2	m=3	m=4	m=5
na=nb=1	4.39547E-07	3.92606E-07	3.78788E-07	3.78669E-07	3.78591E-07
na=nb=2	1.75637E-08	4.53709E-10	1.85536E-10	2.01172E-10	2.43587E-10
na=nb=3	2.15424E-09	1.72534E-10	1.48096E-10	2.79738E-10	4.55176E-10
na=nb=4	1.17815E-09	1.39596E-10	1.80415E-10	3.91906E-10	3.69505E-10
na=nb=5	1.24867E-09	1.38934E-10	1.89118E-10	5.52253E-10	6.8042E-10

04

Conclusion

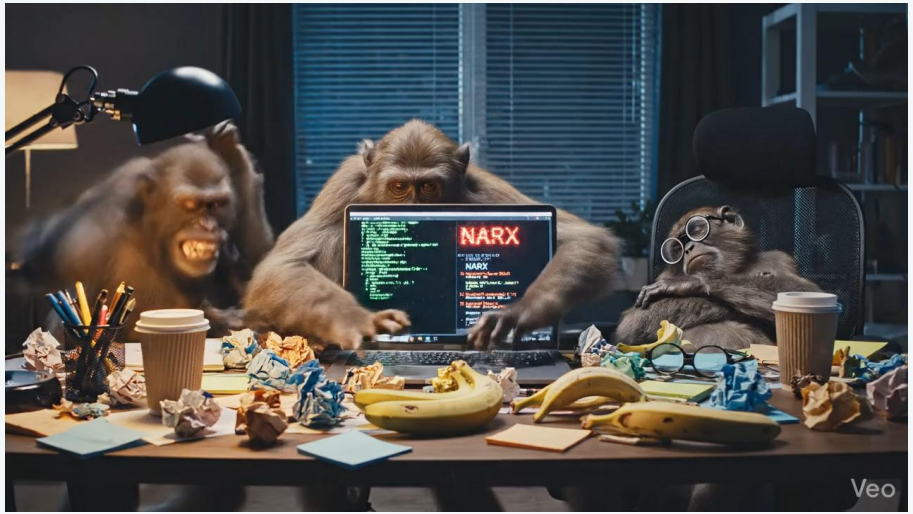
Conclusion

After studying the system, we observe that increasing the polynomial degree and the orders na = nb improves the accuracy of the approximate model, resulting in a smaller error, up to the point where overfitting occurs.

Our brains while working on this project:

Link to video:

- https://youtu.be/nBN85clXWC8



Bibliography

Informations:

- lab5_arx_en.pdf
- System Identification - Control Engineering EN, 3rd year B.Sc. Technical University of Cluj-Napoca Romania

Slides template:

- Free Google Slides themes and Powerpoint templates | Slidesgo

Thank you for your attention!